

Command lab

Objetivo

Preparar um pequeno sistema de pedidos de cafeteria usando o padrão **Command**, seguindo a estrutura apresentada em aula.

O objetivo é mostrar que uma requisição pode ser encapsulada em um objeto, desacoplando quem solicita a execução de quem realmente executa a operação.

Contexto

Considere uma cafeteria simples:

- o cliente faz um pedido;
- o garçom recebe o pedido;
- o cozinheiro prepara o pedido.

O garçom **não sabe cozinhar**.

Ele apenas recebe um objeto do tipo Command e manda executar.

O sistema também deverá permitir cancelar o **último pedido executado**, usando um `undo()` simples.

Diagrama de referência

O sistema deve seguir a estrutura discutida em aula:

Papel no padrão	Classe no sistema
Command	Command
ConcreteCommand	PedidoCafe, PedidoCha
Receiver	Cozinheiro
Invoker	Garcom
Client	classe principal

Fluxo conceitual:

Client → Garcom → Command → Cozinheiro

Parte 1 — Interface Command

Implemente a interface:

interface Command

Métodos esperados:

void execute();

void undo();



Parte 2 — Receiver

Implemente a classe:

Cozinheiro

Métodos esperados:

prepararCafe()

cancelarCafe()

prepararCha()

cancelarCha()



Parte 3 — Commands concretos

Implemente as classes:

PedidoCafe

PedidoCha

Cada classe deve:

- implementar Command;
- possuir uma referência para Cozinheiro;
- chamar o método correto do cozinheiro em execute();
- chamar o método inverso em undo()

Exemplo:

PedidoCafe.execute() → cozinheiro.prepararCafe()

PedidoCafe.undo() → cozinheiro.cancelarCafe()

Parte 4 — Invoker

Implemente a classe:

Garcom

Atributos esperados:

pedido: Command

ultimoPedido: Command

Métodos esperados:

setPedido(Command pedido)

levarPedido()

cancelarUltimoPedido()

 Regra importante:

O Garcom deve conhecer apenas a interface Command.

Ele não deve conhecer diretamente PedidoCafe, PedidoCha ou Cozinheiro.

Parte 5 — Preparação do exemplo para aula

Prepare uma classe principal que demonstre:

1. criação do cozinheiro;
2. criação do garçom;
3. criação de um pedido de café;
4. execução do pedido de café;
5. cancelamento do pedido de café;
6. criação de um pedido de chá;
7. execução do pedido de chá;
8. cancelamento do pedido de chá.

O que deve ser entregue

O aluno deverá entregar:

- diagrama UML de classes;
- código preparado para apresentação em aula.

 **Não incluir a classe principal no diagrama UML.**

Elementos obrigatórios do diagrama

O diagrama deve conter:

1. Interface

Command

Métodos:

execute(): void
undo(): void

2. Classe

Cozinheiro

Métodos:

prepararCafe(): void
cancelarCafe(): void
prepararCha(): void
cancelarCha(): void

3. Classe

PedidoCafe

Deve implementar Command.

Atributo:

cozinheiro: Cozinheiro

Métodos:

execute(): void
undo(): void

. Classe

PedidoCha

Deve implementar Command.

Atributo:

cozinheiro: Cozinheiro

Métodos:

execute(): void
undo(): void

. Classe

Garcom

Atributos:

pedido: Command
ultimoPedido: Command

Métodos:

setPedido(pedido: Command): void
levarPedido(): void
cancelarUltimoPedido(): void



Relações que devem aparecer no diagrama

O diagrama deve representar explicitamente:

PedidoCafe implementa Command
PedidoCha implementa Command
Garcom usa Command
PedidoCafe usa Cozinheiro
PedidoCha usa Cozinheiro

Perguntas de reflexão

1. Quem é o Receiver no sistema?
2. Quem é o Invoker?
3. Quem são os ConcreteCommands?
4. Onde ocorre o desacoplamento?
5. Por que o Garcom não precisa conhecer diretamente os métodos do Cozinheiro?
6. Qual é o papel do método undo()?
7. Por que este exemplo representa o padrão Command?



Restrições

- O Garcom deve trabalhar apenas com objetos do tipo Command.
- Não usar if ou switch para decidir qual pedido executar.
- Não usar listas de comandos.
- Não implementar histórico de comandos.
- O undo() deve cancelar apenas o último pedido executado.